

Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (Raise Hand).

When instructor calls on you, click microphone icon to unmute. When you've finished speaking, *be sure to mute yourself again.*

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class for their own private use.

Releasing these recordings is strictly prohibited.

SEAS 8515

Data Engineering for AI

Professor Steven Cocke
Week 4: June 6, 2026

Agenda

- Week 3 Assignment Review
- Data Storage & Interaction Continued
- Distributed Computing Concepts
- Spark
- Assignment Overview for This Week

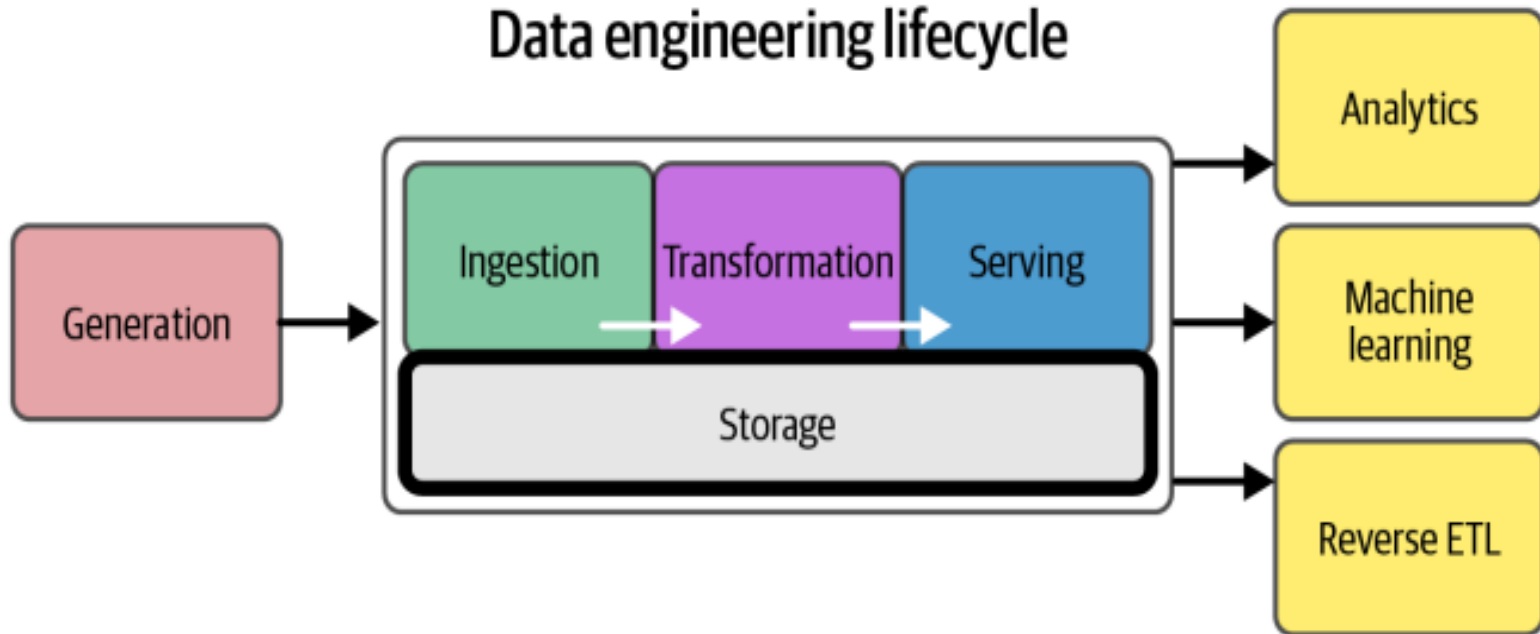
THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Week 3 Assignment Overview

Data Storage: Structures & Interaction Continued

Data engineering lifecycle



Undercurrents:



Data Storage Engines & Processing

Databases optimized for

Transactional
processing

Analytical
processing

OnLine Transaction Processing (OLTP)

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
 - Insert when generated
 - Occasional update/delete

OnLine Transaction Processing

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
 - Inserted when generated
 - Occasional update/delete
- Requirements
 - Low latency
 - High availability

OnLine Transaction Processing

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
 - Operations:
 - Inserted when generated
 - Occasional update/delete
 - Requirements
 - Low latency
 - High availability
 - Full ACID not always necessary
 - **Atomicity**: all the steps in a transaction fail or succeed as a group
 - If payment fails, don't assign a driver
 - **Isolation**: concurrent transactions happen as if sequential
 - Don't assign the same driver to two different requests that happen at the same time
- ACID** describes a set of properties that guarantee reliable transactions in databases.
- **Atomicity** – A transaction is all-or-nothing
 - **Consistency** – The database moves from one valid state to another
 - **Isolation** – Concurrent transactions do not interfere with each other
 - **Durability** – Once committed, data persists even after failures

OnLine Transaction Processing

- Transactions: tweeting, ordering a Lyft, uploading a new model, etc.
- Operations:
 - Inserted when generated
 - Occasional update/delete
- Requirements
 - Low latency
 - High availability
- Typically row-major

Row

```
INSERT INTO RideTable(RideID, Username, DriverID, City, Month, Price)
VALUES ('10', 'memelord', '3932839', 'Stanford', 'July', '20.4');
```

OnLine Analytical Processing (OLAP)

- How to get aggregated information from a large amount of data?
 - e.g. what's the average ride price last month for riders at Stanford?
- Operations:
 - Mostly SELECT

OnLine Analytical Processing

- Analytical queries: aggregated information from a large amount of data?
 - e.g. what's the average ride price last month for riders at Stanford?
- Operations:
 - Mostly SELECT
- Requirements:
 - Can handle complex queries on large volumes of data
 - Okay response time (seconds, minutes, even hours)

OnLine Analytical Processing

- Analytical queries: aggregated information from a large amount of data?
 - e.g. what's the average ride price last month for riders at Stanford?
- Operations:
 - Mostly SELECT
- Requirements:
 - Can handle complex queries on large volumes of data
 - Okay response time (seconds, minutes, even hours)
- Typically column-major

Column

```
SELECT AVG(Price)
FROM RideTable
WHERE City = 'Stanford' AND Month = 'July';
```

OLTP & OLAP are outdated terms

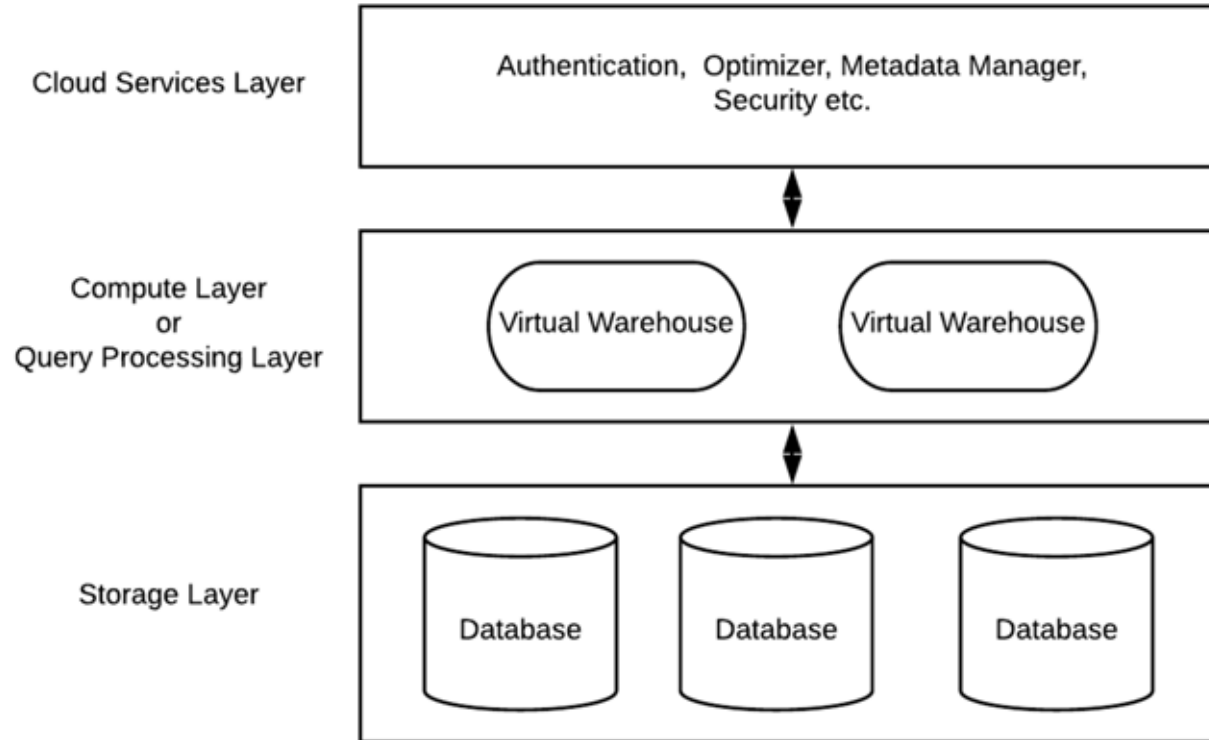


Decoupling storage & processing

- OLTP & OLAP: how data is stored is also how it's processed
 - Same data being stored in multiple databases
 - Each uses a different processing engine for different query types
 - **How data is stored dictates how it is processed**
- New paradigm: storage is decoupled from processing
 - Data can be stored in the same place (object storage like S3)
 - Storage is cheap, durable, and shared
 - A processing layer on top that can be optimized for different query types (SQL engines, spark, ML pipelines, etc)



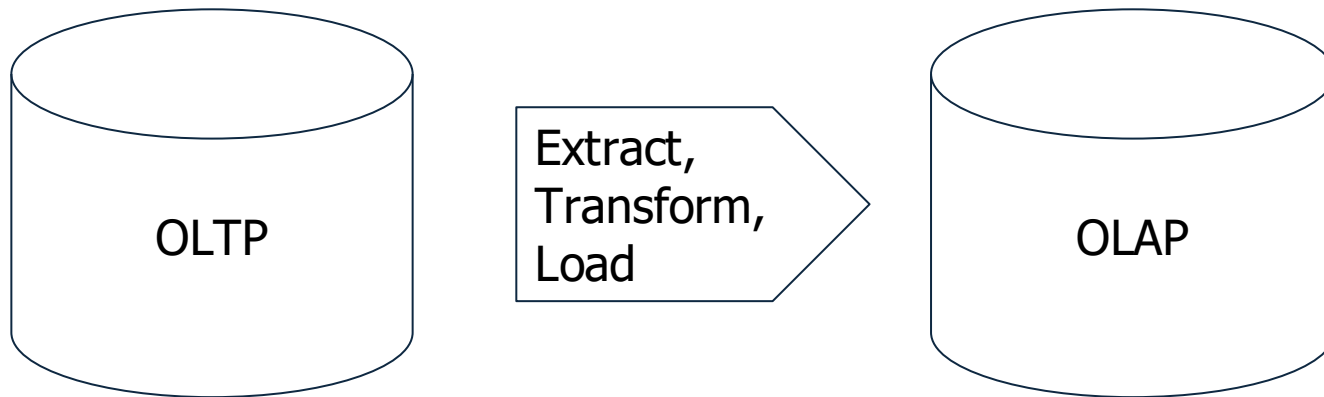
Decoupling storage & processing



[Source](#)

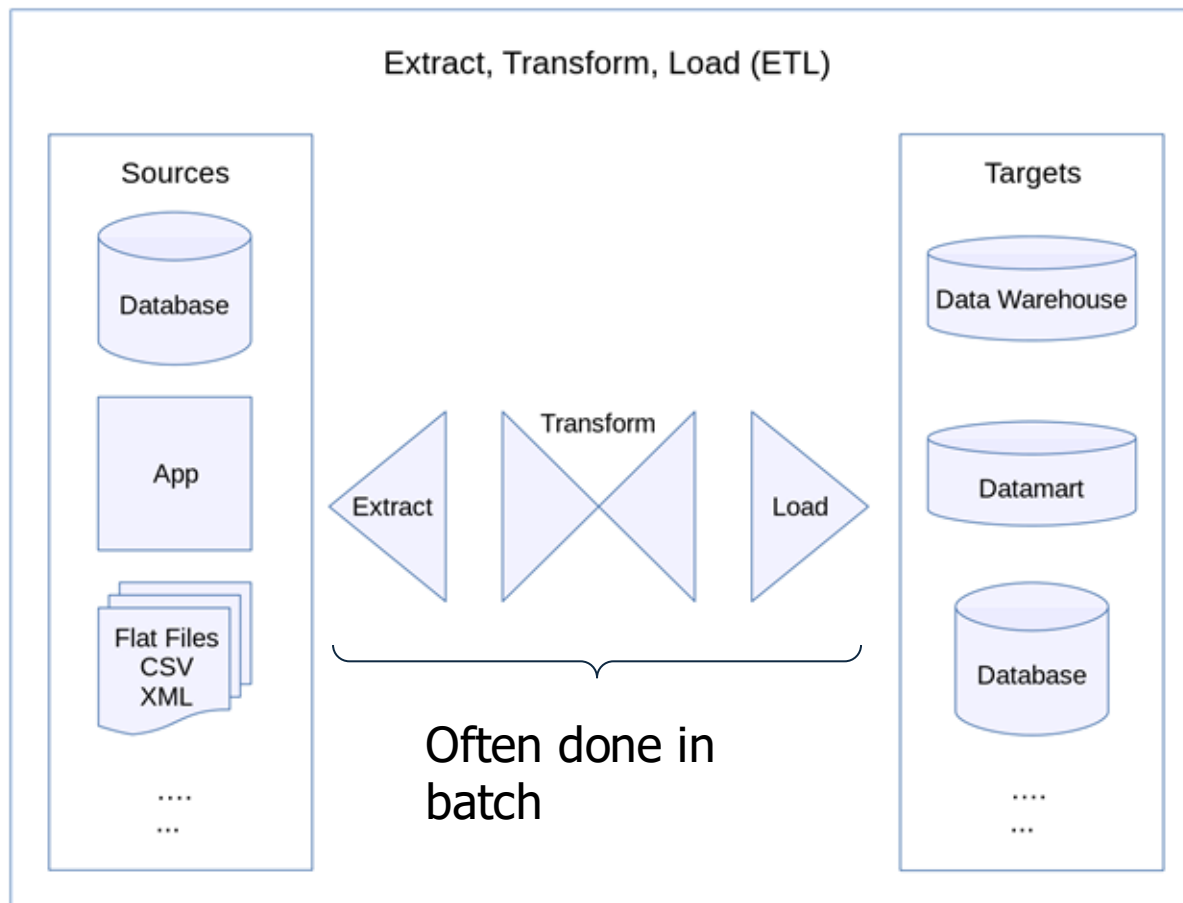


ETL (Extract, Transform, Load)



Transform: the meaty part

- cleaning, validating, transposing, deriving values, joining from multiple sources, deduplicating, splitting, aggregating, etc.



[Source](#)

ETL Adapting

Traditional Model

- OLTP and OLAP used different databases
- Data had to be copied and transformed
- ETL existed to move data from transactional systems into analytical systems
- Example flow:
 - OLTP DB → ETL → OLAP DB
- ETL was required because storage and processing were bound together.

Modern Model (Decoupled Storage & Compute)

- Data is stored once (data lake / object storage)
- Multiple processing engines query the same data
- ETL shifts from:
 - *Moving data between databases*
 - *To preparing and structuring data* for different workloads
- Example flow:
 - OLTP DB → ETL → Data Lake → Multiple Engines

What is a file?

- A file is a data entity with specific read, write, and reference characteristics used by software and operating systems. We define a file to have the following characteristics:
 - **Finite length**
 - A file is a finite-length stream of bytes.
 - **Append operations**
 - We can append bytes to the file up to the limits of the host storage system.
 - **Random access**
 - We can read from any location in the file or write updates to any location.

Usually File storage systems organize files into a directory tree.

File Formats (Parquet)

- it's cross platform
- it's a recognized file format used by many systems
- it stores data in a column layout
- it stores metadata

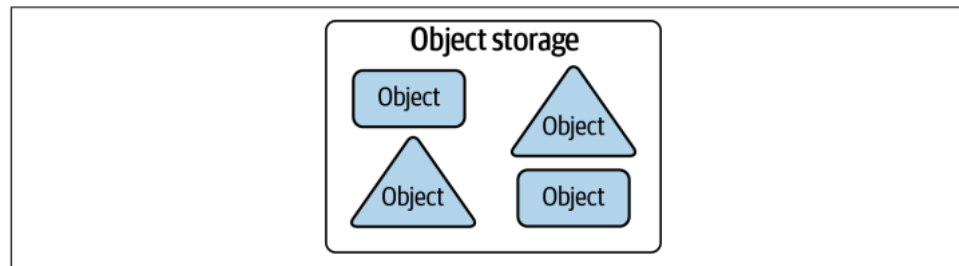


Basis for Delta lake... will be discussed shortly

Src: <https://www.jumpingrivers.com/blog/parquet-file-format-big-data-r/>

What is an object?

- The term object storage is somewhat confusing because object has several meanings in computer science. In this context, we're talking about a specialized file-like construct. It could be any type of file—TXT, CSV, JSON, images, videos, or audio.



Object Storage Characteristics

- One of the most attractive characteristics of cloud object storage is that it is straightforward to manage and use.
- Object storage was arguably one of the first “serverless” services; engineers don’t need to consider the characteristics of underlying server clusters or disks.
- An object store is a key-value store for immutable data objects. We lose much of the writing flexibility we expect with file storage on a local disk in an object store.
- Objects don’t support random writes or append operations; instead, they are written once as a stream of bytes.

Object Storage

- After this initial write, objects become immutable.
- To change data in an object or append data to it, we must rewrite the full object. Object stores generally support random reads through range requests, but these lookups may perform much worse than random reads from data stored on an SSD.
- Compared to local random access file storage, object stores don't need to support locks or change synchronization, allowing data storage across massive disk clusters.

Object Storage

- Object stores support extremely performant parallel stream writes and reads across many disks (Transparent).
- In a cloud environment, write / read speed scale based on quotas/available capacity.
- object storage ideal for serving high-volume web traffic or delivering data to highly parallel distributed query engines.
- Typical cloud object stores save data in several availability zones

Object Storage

- Object stores are not good for updates, but this is only partially true. Object stores are an inferior fit for transactional workloads (better use transactional DBs)
- Object stores are now the gold standard of storage for data lakes.

Amazon S3

- Amazon Simple Storage Service (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance.
- Customers of all sizes and industries can use Amazon S3 to store and protect any amount of data for a range of use cases, such as data lakes, websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics.

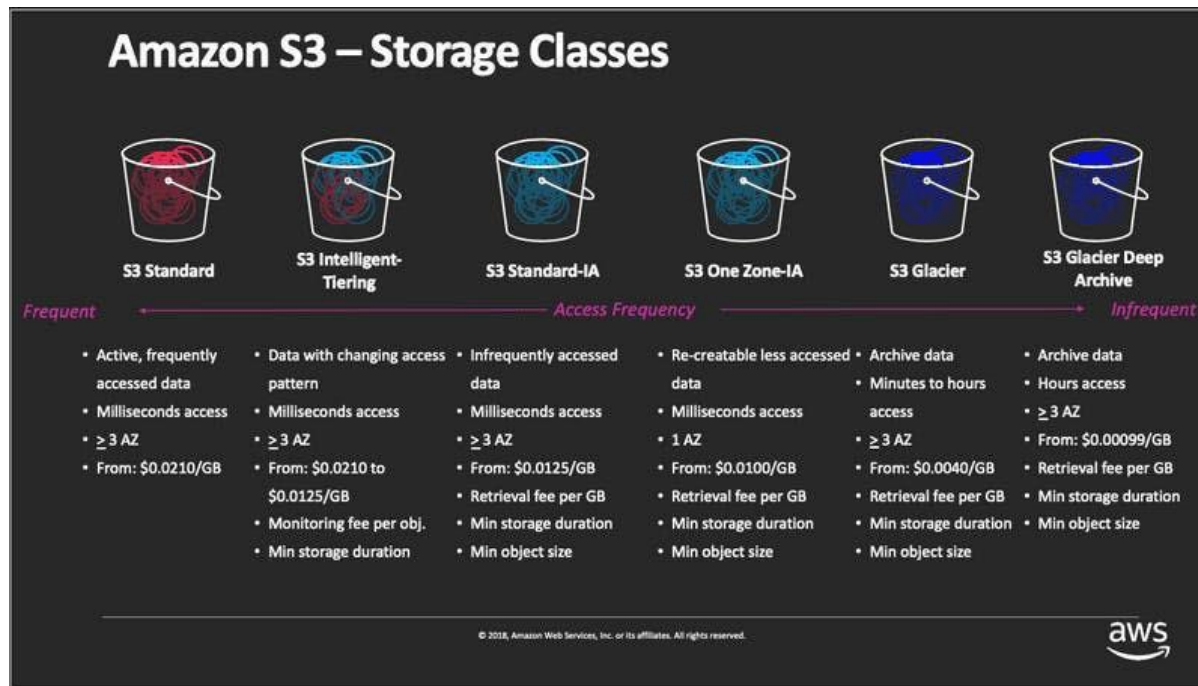
Amazon S3



Amazon S3

Storage classes:

Amazon S3 offers a range of storage classes designed for different use cases. For example, you can store mission-critical production data in S3 Standard or S3 Express One Zone for frequent access.



Src: amazon; <https://blog.jineshkumar.com/all-about-aws-s3-storage>

Amazon S3

AWS S3 STORAGE CLASSES					 Whizlabs Success, certified!
Features	S3-Standard	S3-Standard IA	S3 One Zone IA	S3 - RRS	Glacier
Durability	99.999999999%	99.999999999%	99.999999999% (Single AZ)	99.99%	99.999999999%
Availability	99.99%	99.9%	99.5%	99.99%	NA
Availability Zones	>=3	>=3	1	1	>=3
Minimum Storage Duration	Unlimited	30 Days	30 Days	Unlimited	90 Days
Access Speed (Latency)	Miliseconds	Miliseconds	Miliseconds	Miliseconds	1 min to 12 hrs
Data Retrieval Fee	Nil	Per GB Data Retrived	Per GB Data Retrived	Nil	Per GB Data Retrived
Minimum Billable Object Size	NA	128 KB (the smaller objects will be charged for 128KB)	128 KB (the smaller objects will be charged for 128KB)	NA	NA
Type of Storage	Object	Object	Object	Object	Object
Lifecycle Management Policies	Available	Available	Available	Available	Available
SSL Support	Yes	Yes	Yes	Yes	Yes
Amazon S3 SLA Backing	99.9%	99%	99%	Data Not Available	NA

Src: amazon; <https://www.whizlabs.com/blog/aws-s3/>

Amazon S3 – Storage Management

- **S3 Lifecycle** – Configure a lifecycle configuration to manage your objects and store them cost effectively throughout their lifecycle. You can transition objects to other S3 storage classes or expire objects that reach the end of their lifetimes.
- **S3 Object Lock** – Prevent Amazon S3 objects from being deleted or overwritten for a fixed amount of time or indefinitely (regulatory/extra protection).
- **S3 Replication** – Replicate objects and their respective metadata and object tags to one or more destination buckets in the same or different AWS Regions for reduced latency, compliance, security, and other use cases.

Object Storage vs. File Storage

- In traditional block storage systems, data is divided into blocks (e.g., 512 KB or 4 KB in size), with the disk controller managing the addresses and locations of each block.
- S3, as an object storage system, abstracts away blocks and sectors. It does not treat data as a series of fixed-size blocks, but rather as entire objects that can vary in size, up to a maximum of 5 TB for a single object.

How S3 Works

- Amazon S3 is an object storage service that stores data as objects within buckets.

An object is a file and any metadata that describes the file.

- A bucket is a container for objects. You can store any number of objects in a bucket and can have up to 100 buckets in your account.
- To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region. Then, you upload your data to that bucket as objects in Amazon S3. Each object has a key(or key name), which is the unique identifier for the object within the bucket.

How S3 Works

- S3 provides features that you can configure to support your specific use case. For example, you can use S3 Versioning to keep multiple versions of an object in the same bucket, which allows you to restore objects that are accidentally deleted or overwritten.
- Buckets and the objects in them are private and can be accessed only if you explicitly grant access permissions. You can use bucket policies, AWS Identity and Access Management (IAM) policies, access control lists (ACLs), and S3 Access Points to manage access.

Objects

- Objects consist of object data and metadata. The metadata is a set of name-value pairs that describe the object. These pairs include some default metadata, such as the date last modified, and standard HTTP metadata, such as Content-Type
- An object is uniquely identified within a bucket by a key (name) and a version ID (if S3 Versioning is enabled on the bucket).

Keys

- An object key (or key name) is the unique identifier for an object within a bucket. Every object in a bucket has exactly one key. The combination of a bucket, object key, and optionally, version ID (if S3 Versioning is enabled for the bucket) uniquely identify each object. So you can think of Amazon S3 as a basic data map between "bucket + key + version" and the object itself.

Regions

- You can choose the geographical AWS Region where Amazon S3 stores the buckets that you create.
- You might choose a Region to optimize latency, minimize costs, or address regulatory requirements.
- Objects stored in an AWS Region never leave the Region unless you explicitly transfer or replicate them to another Region

Amazon Data Consistency Model

- Amazon S3 provides strong read-after-write consistency for PUT and DELETE requests of objects in your Amazon S3 bucket in all AWS Regions.
- This behavior applies to both writes to new objects as well as PUT requests that overwrite existing objects and DELETE requests. In addition, read operations on Amazon S3 Select, Amazon S3 access controls lists (ACLs), Amazon S3 Object Tags, and object metadata (for example, the HEAD object) are strongly consistent.
- Updates to a single key are atomic.
- Amazon S3 achieves high availability by replicating data across multiple servers within AWS data centers

Strong Consistency

- **Assurance of Correctness:** Strong consistency is essential in applications where it's critical that every read operation returns the most recent write. This is crucial for scenarios where correctness is non-negotiable, like financial transactions, inventory management, or critical system logs.
- **Data Integrity Across Nodes:** In systems like Amazon S3, strong consistency ensures that data reads immediately reflect any successful write or update, regardless of which node is serving the read request.
- **Drawbacks:** Strong consistency often requires **synchronization across nodes** or **quorum-based reads and writes**.

Eventual Consistency

- **High Availability and Low Latency:** Eventual consistency provides high availability and lower latency by allowing nodes to respond to read and write requests independently without waiting for synchronization. This is particularly advantageous in systems where scalability and performance are prioritized.
- **Trade-Off with Stale Reads:** there's a possibility of reading outdated data immediately after a write, as updates propagate asynchronously. However, it's guaranteed that given enough time without further updates, all nodes will converge to the latest value.
- **Use Cases:** Eventual consistency is suitable for applications where absolute correctness is not required immediately, such as social media feeds, caching layers, and some content delivery networks (CDNs)



Hands On.....

Amazon S3

What is a Data Lake?

A data lake is a centralized repository that stores large volumes of raw data in its native format.

Key characteristics

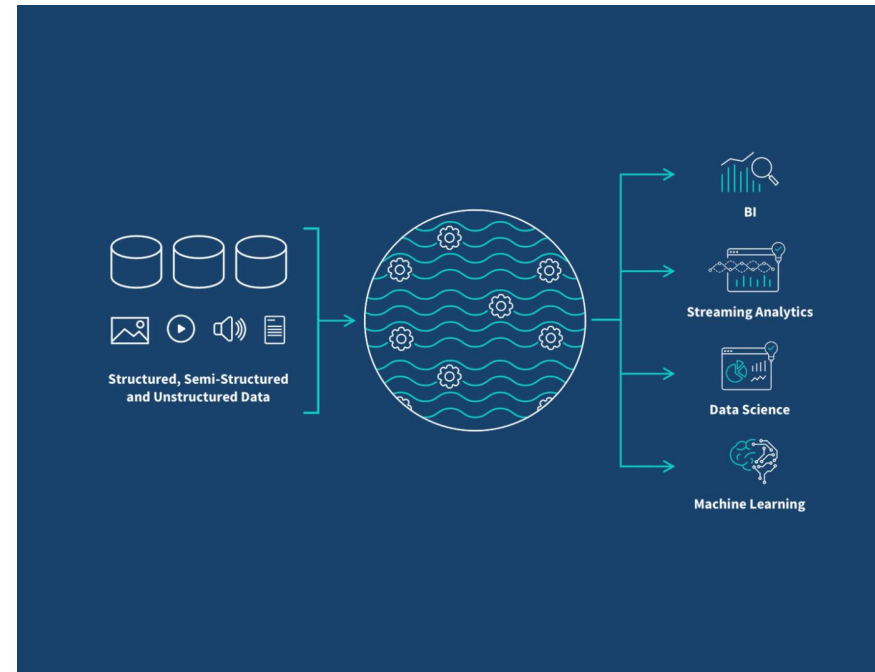
- Stores raw, semi-structured, and unstructured data
- Uses schema-on-read (structure applied at query time)
- Built on object storage (S3, ADLS, GCS)
- Scales cheaply and elastically
- Supports batch analytics, streaming, and ML workloads

Strengths

- High flexibility for evolving data and use cases
- Low storage cost
- Fast ingestion with minimal upfront modeling

Challenges

- Data quality is not enforced by default
- Governance and discoverability require discipline
- Risk of becoming a “data swamp”
- Performance depends heavily on file format and tooling



<https://www.qlik.com/us/data-lake>

What is a Data Warehouse?

A data warehouse is a structured system optimized for analytics, reporting, and business intelligence.

Key characteristics

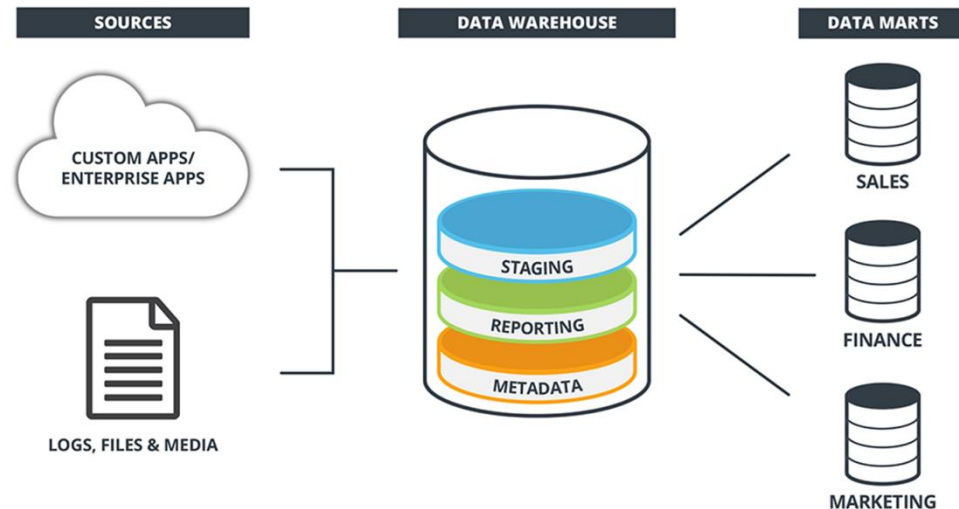
- Stores curated, structured data
- Uses schema-on-write (data modeled before ingestion)
- Optimized for SQL analytics and BI tools
- Strong performance, consistency, and governance
- Typically uses columnar storage and indexing

Strengths

- Fast and predictable query performance
- Strong data quality guarantees
- Easy for analysts and business users
- Excellent support for dashboards and KPIs

Challenges

- Higher storage and compute cost
- Less flexible for new data types
- Requires upfront schema design
- Not ideal for raw or unstructured data



<https://www.astera.com/type/blog/data-warehouse-use-definition/>

What is a Data Lakehouse?

A data lakehouse is an architecture that combines the flexibility of a data lake with the reliability and performance of a data warehouse.

Core idea

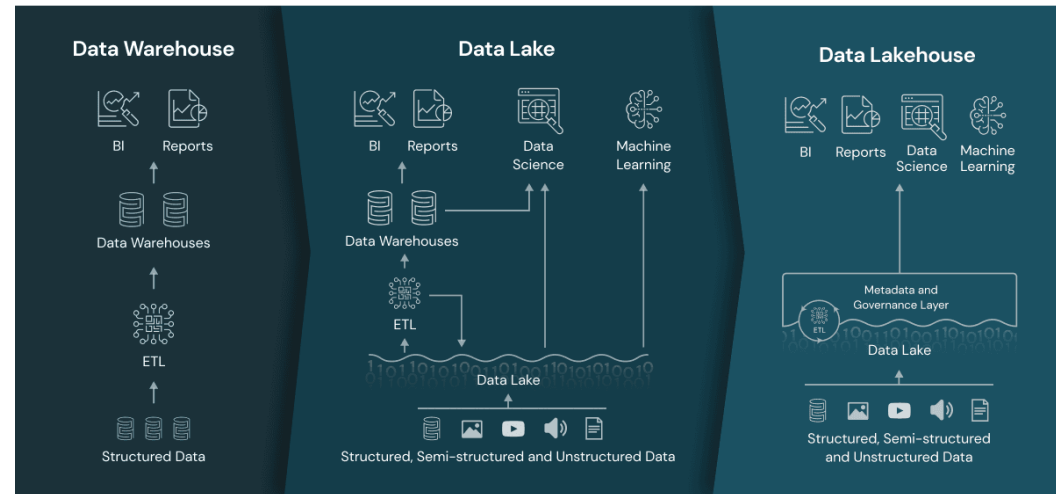
- Store data in low-cost object storage (like a data lake)
- Add warehouse-like features on top via a table format and transaction layer

Key characteristics

- Built on object storage (S3, ADLS, GCS)
- Uses open file formats (Parquet)
- Adds ACID transactions
- Supports schema enforcement AND schema evolution
- Enables SQL analytics, BI, and ML on the same data
- Eliminates duplicate lake + warehouse copies

What it fixes

- Data lakes → lack transactions, governance, reliability
- Data warehouses → expensive, rigid, poor for ML/raw data



<https://www.databricks.com/glossary/data-lakehouse>

Data Lake vs. Data Warehouse

Dimension	Data Lake	Data Warehouse
Data type	Raw, semi-structured, unstructured	Structured, curated
Schema	Schema-on-read	Schema-on-write
Storage	Object storage	Managed analytical storage
Cost	Low storage cost	Higher storage cost
Flexibility	Very high	Limited
Query performance	Variable	High and predictable
Governance	Manual / evolving	Strong, built-in
Typical users	Data engineers, data scientists	Analysts, business users
Common use cases	ML, exploration, streaming	BI, reporting, KPIs
Examples	Delta Lake, Apache Iceberg, Apache Hudi, raw Parquet/CSV on S3	Snowflake, Google BigQuery, Amazon Redshift, Azure Synapse Analytics

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Distributed Computing Concepts

Traditional Large-Scale Computation

Early Days: Processor Bound, and Memory Bound (the CPUs were “slower”...)

Early Solution: Bigger Machine... Faster CPUs, More memory

Aspect	Mainframes	Exadata
Era	Introduced in the 1950s-60s (e.g., IBM).	Introduced in 2008 as a modern solution.
Scalability	Vertical scaling only.	Vertical scaling but integrates modern tech.
Task Diversity	General-purpose computing.	Specialized for database workloads.
Technology Stack	Proprietary legacy systems.	Combines modern hardware and software layers.

Distributed Systems

“In pioneer days they used oxen for heavy pulling, and when one ox couldn’t budge a log, we didn’t try to grow a larger ox. We shouldn’t be trying for bigger computers, but for more systems of computers.”

– Grace Hopper



Distributed = many computers tackling a (single) workload.

Key Distributed Computing Concepts

Distributed computing is a model where computation and data are spread across multiple machines that work together as a single system.

Core idea

- Break large problems into smaller tasks
- Execute tasks in parallel
- Aggregate results into a final output

Why distributed computing exists

- Data sizes exceed single-machine capacity
- Performance limits of CPUs, memory, and disks
- Need for scalability and reliability

Core Distributed Computing Concepts

- **Node:** an individual machine
- **Cluster:** a group of nodes
- **Parallelism:** tasks run simultaneously
- **Data locality:** move computation to data
- **Fault tolerance:** system survives failures

Why Distributed Systems Need Specialized Frameworks

Challenges without a framework

- Manual data partitioning
- Manual failure recovery
- Manual scheduling
- Manual communication between machines

Distributed frameworks handle

- Task scheduling
- Data movement
- Failure recovery
- Resource management

Hadoop was one of the first widely adopted frameworks to solve these problems at scale.

What is Hadoop?

Hadoop is an open-source framework for distributed storage and batch processing of large datasets.

Original design goals

- Run on commodity hardware
- Expect hardware failures
- Scale linearly by adding machines

Hadoop separates concerns:

- Storage
- Resource management
- Processing

Core Hadoop Components

Hadoop consists of three primary components:

- **HDFS** -> Distributed storage
- **YARN** -> Cluster resource management
- **MapReduce** -> Batch processing engine

Each component has a single responsibility.

HDFS: Hadoop Distributed File System

HDFS is a distributed file system designed for storing very large files reliably across many machines.

- Optimized for high-throughput, sequential access
- Write-once, read-many model
- Fault tolerant by design
- Designed for large files, not small ones

An HDFS cluster consists of:

- One active NameNode (master)
- Many DataNodes (workers)
- Optional Standby NameNode for high availability
- Metadata → NameNode
- Actual data → DataNodes

NameNode: The Metadata Manager

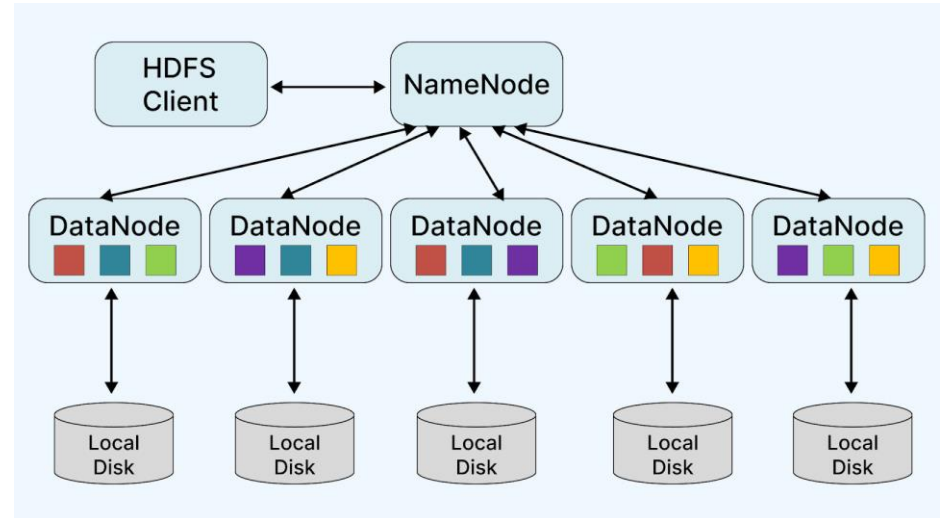
The NameNode manages the HDFS namespace and metadata.

Responsibilities

- Maintains directory structure
- Tracks file permissions and ownership
- Maps:
 - files → blocks
 - blocks → DataNodes

Important!!

- Stores no actual file data
- Critical component of the cluster



<https://www.upgrad.com/blog/hadoop-clusters-overview/>

DataNodes: The Storage Workers

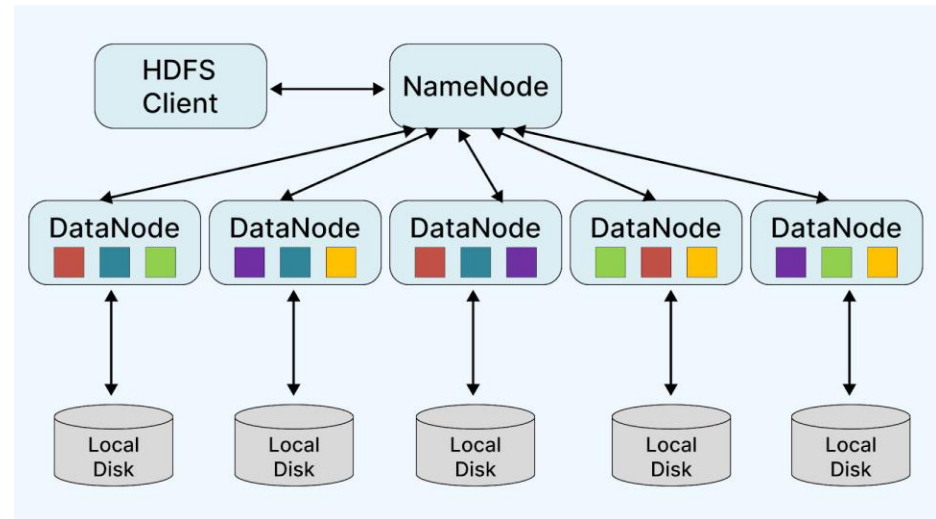
DataNodes store the actual data blocks.

Responsibilities

- Store blocks on local disks
- Serve read/write requests
- Send heartbeats to the NameNode
- Report block status

Failure handling

- If a DataNode fails, blocks are re-replicated elsewhere



HDFS Namespace

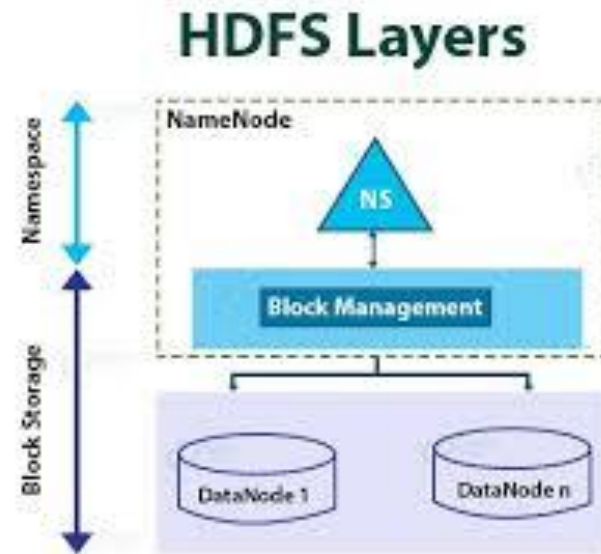
The HDFS namespace is the logical filesystem view.

Includes:

- Directories
- Files
- Metadata (permissions, ownership)

Main idea

- Namespace exists only in the NameNode
- DataNodes know nothing about directories or filenames



<https://data-flair.training/blogs/hadoop-hdfs-federation-tutorial/>

The namespace is the logical structure of the filesystem (files, directories, and paths), while the NameNode is the service that stores, manages, and serves that namespace and its metadata.

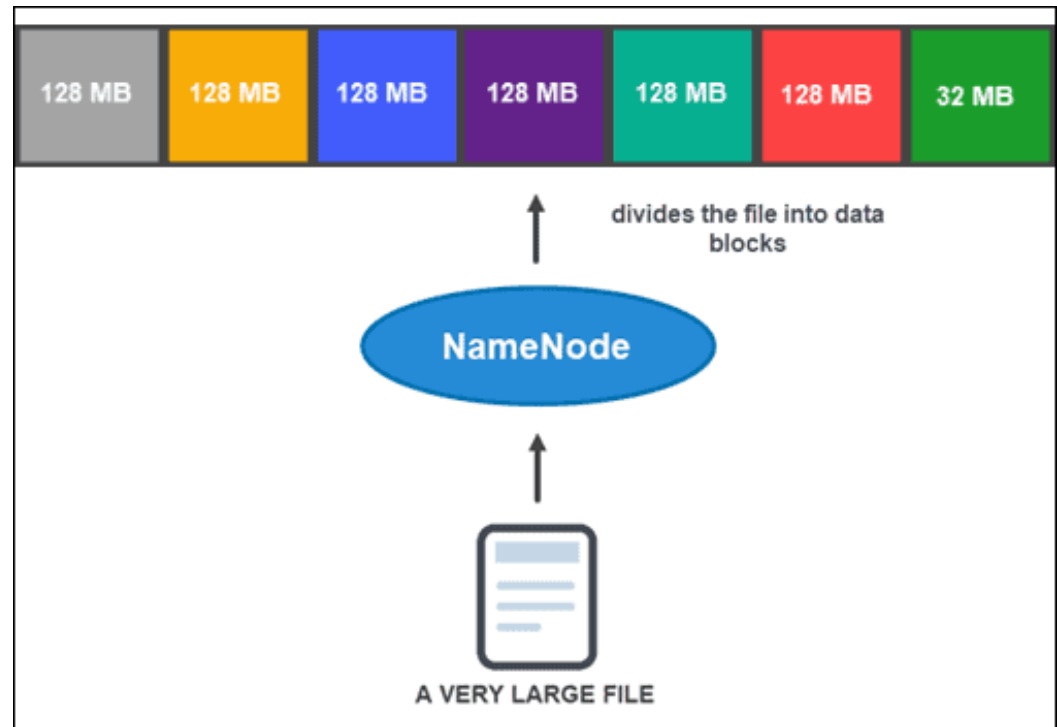
HDFS Block Storage Model

How files are stored

- Files are split into large blocks (e.g., 128 MB)
- Blocks are distributed across DataNodes

Why large blocks

- Higher throughput
- Efficient for large-scale processing

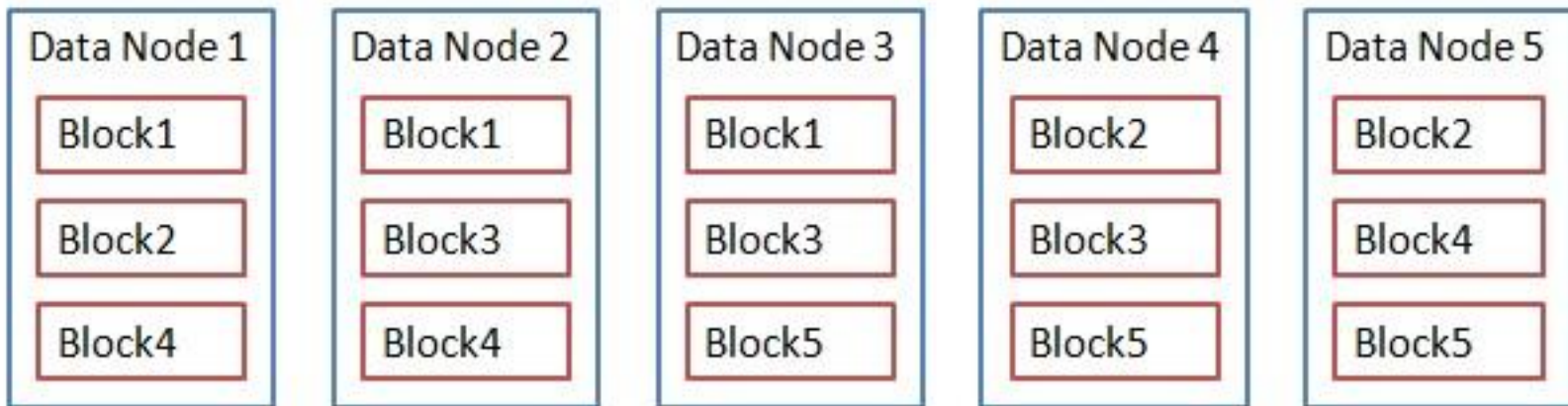
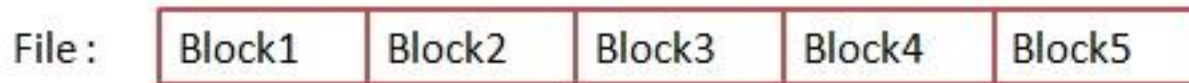


<https://phoenixnap.com/kb/what-is-hdfs>

Replication in HDFS

HDFS uses replication for fault tolerance.

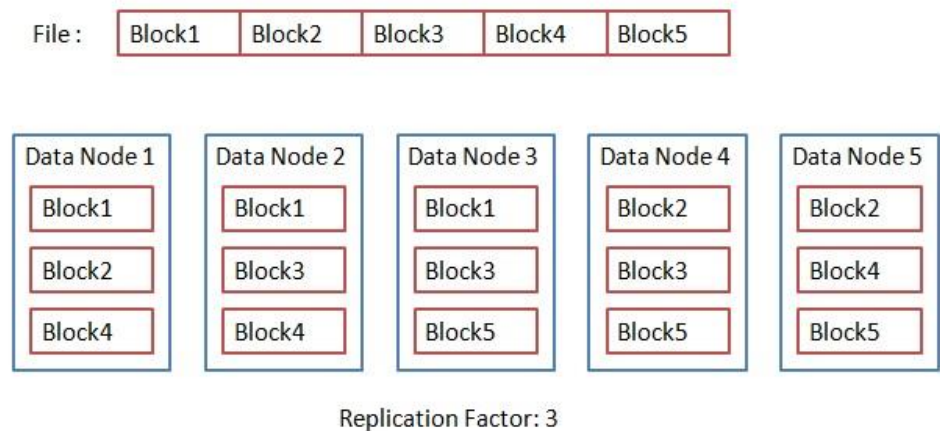
- Default replication factor: **3**
- Each block is stored on multiple DataNodes
- Ensures data availability if nodes fail



Replication Factor: 3

Replication in HDFS

- When a file is replicated in HDFS, each block of the file is stored on multiple DataNodes, but the NameNode tracks exactly where every replica lives. When a client (or a processing engine like MapReduce or Spark) needs to read or process the file, it first asks the NameNode for block locations, then reads the closest available replica, typically one on the same node or rack.
- For processing, the framework schedules computation where the data already exists (data locality), so different replicas of different blocks can be processed in parallel across the cluster. If a DataNode fails or is slow, HDFS automatically redirects reads to another replica, ensuring fault tolerance without the application needing to handle failures.



Rack Awareness

Rack awareness places replicas across different physical racks.

Why this matters

- Protects against rack-level failures
- Improves reliability
- Improves read performance

Typical placement

- 1 replica on local rack
- 1 replica on a different rack
- 1 replica on another node in that rack

HDFS High Availability

Early HDFS had a single point of failure (NameNode).

Modern HDFS supports:

- Active NameNode
- Standby NameNode
- Automatic failover

Result

- No single point of failure
- Increased reliability

YARN – Yet Another Resource Negotiator

Apache Hadoop YARN manages cluster resources independently of any specific processing engine.

- It is responsible for allocating CPU and memory across all nodes in the cluster.
- Applications request resources from YARN instead of managing machines directly.

How YARN works

- YARN tracks available resources on each node.
- When a job is submitted, YARN:
 - Decides where it can run
 - Launches containers with allocated CPU and memory
 - Monitors application execution

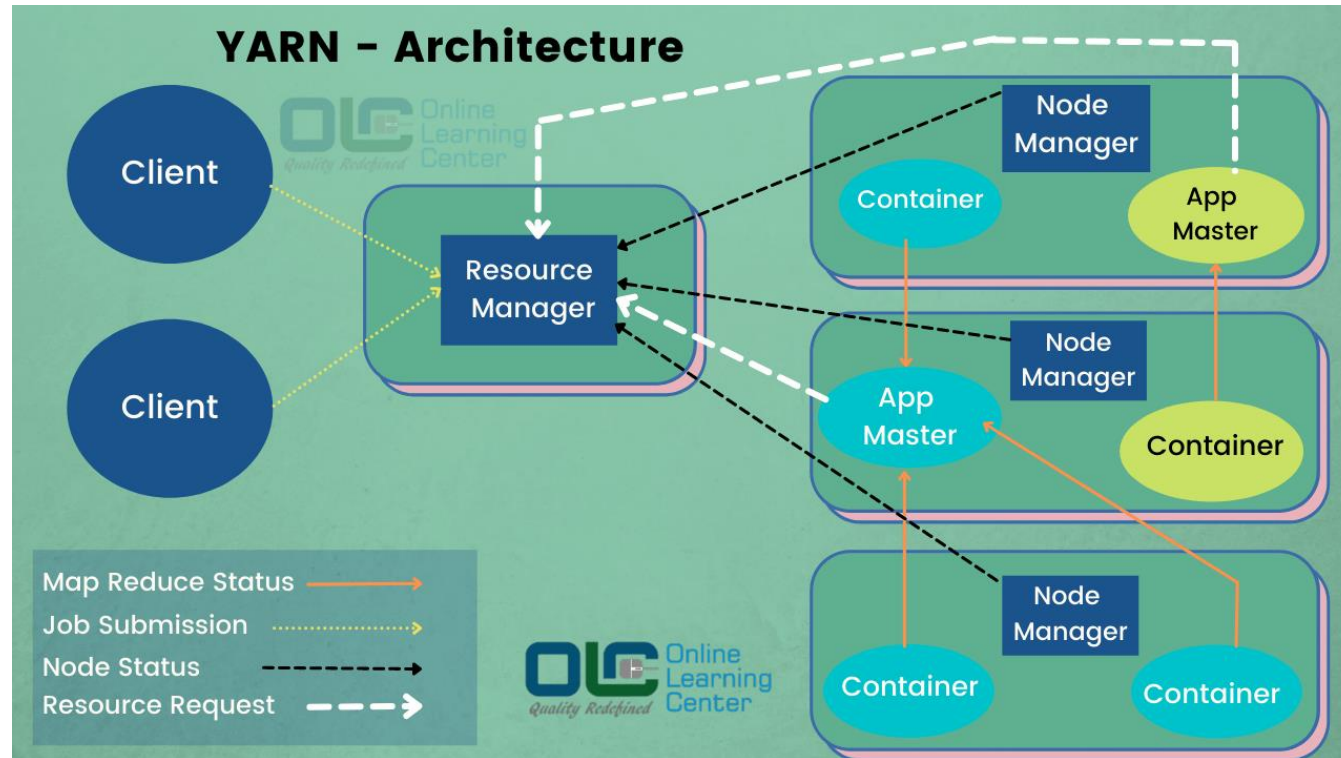
How YARN ties into other components

- HDFS stores the data but does not schedule computation.
- Processing engines (MapReduce, Spark, others) request resources from YARN.
- YARN does not process data. It only manages resources.

Why this separation matters

- Multiple applications can share the same cluster (multi-tenancy).
- Different engines can run side by side.
- Resource management and processing logic evolve independently.

YARN – Yet Another Resource Negotiator



Source

MapReduce: Processing Model

MapReduce is Hadoop's original batch processing engine.

Programming model

1. **Map** — transform input into key–value pairs
2. **Shuffle** — group by key
3. **Reduce** — aggregate values

Strengths

- Simple
- Fault tolerant
- Scales well for batch jobs

Hadoop Components Together

End-to-end flow

- Data stored in HDFS
- Job submitted to YARN
- YARN allocates resources
- MapReduce processes data
- Results written back to HDFS

Design philosophy

- Scale out, not up
- Expect failures
- Move compute to data

The Birth of Spark

Hadoop introduced scalable distributed storage and batch processing, but its processing model had limitations.

What Hadoop did well

- Reliable distributed storage (HDFS)
- Fault tolerance through replication
- Scales across many machines
- Enabled large-scale batch analytics

Where Hadoop (MapReduce) struggled

- Disk I/O between every processing step
- High latency for jobs
- Poor support for iterative algorithms
- Not suitable for interactive analytics

Why Spark emerged

- In-memory computation instead of disk-heavy pipelines
- DAG-based execution (not fixed Map → Reduce)
- Faster for iterative and interactive workloads
- Works with HDFS and cloud object storage
- **Spark kept Hadoop's ideas, but redesigned the execution engine.**

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Spark and Databricks

What is Databricks?

Databricks is a managed data platform that simplifies running Apache Spark and large-scale data workloads in the cloud.

What problem Databricks solves

Running Spark is operationally complex:

- Cluster setup
- Scaling
- Monitoring
- Failures

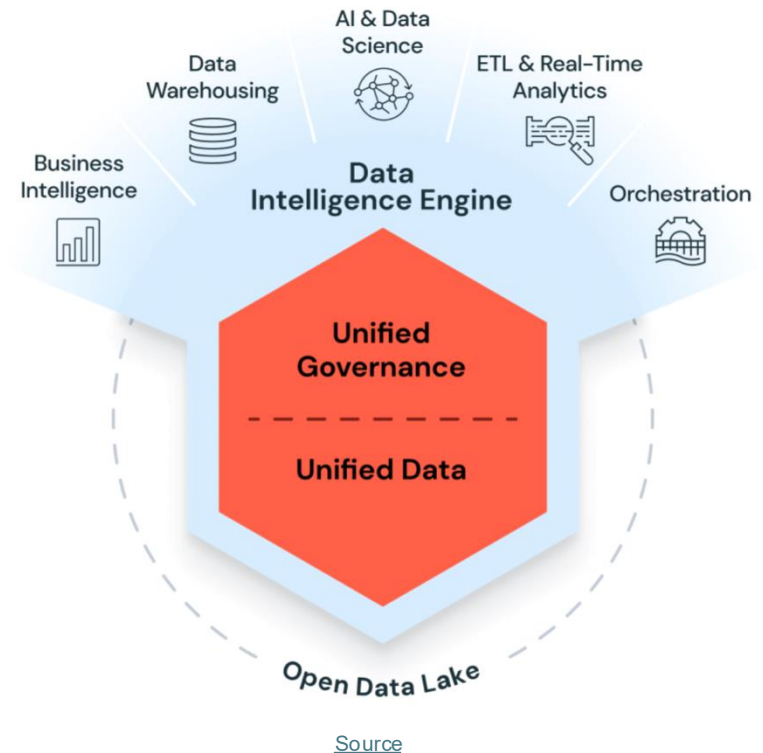
Databricks abstracts this infrastructure away

What Databricks provides

- Managed Spark clusters
- Interactive notebooks (Python, SQL, Scala)
- Job orchestration and scheduling
- Tight integration with cloud object storage
- Governance, access control, and observability

Key distinction

- Spark = compute engine
- Databricks = platform that runs and manages Spark



What is Apache Spark?

Apache Spark is a distributed data processing engine designed for fast, large-scale data analytics.

Why Spark was created

- Hadoop MapReduce was:
 - Disk-based
 - High latency
 - Inefficient for iterative algorithms
- Spark introduced in-memory computation

What Spark enables

- Parallel processing across clusters
- Batch processing
- Streaming analytics
- Machine learning
- SQL analytics



<https://www.databricks.com/spark/about>

Spark Execution Model: Driver, Executors, Tasks

Spark follows a master-worker execution model.

Driver

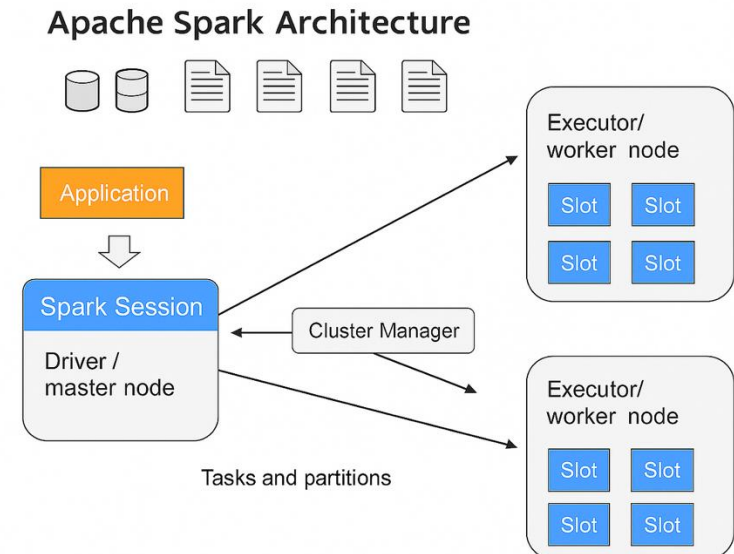
- Runs the user's application
- Builds the execution plan
- Coordinates work across the cluster
- Tracks job progress and failures

Executors

- Run on worker nodes
- Execute tasks assigned by the driver
- Store data in memory or disk
- Return results to the driver

Tasks

- Smallest unit of work
- Operate on a partition of data
- Executed in parallel across executors



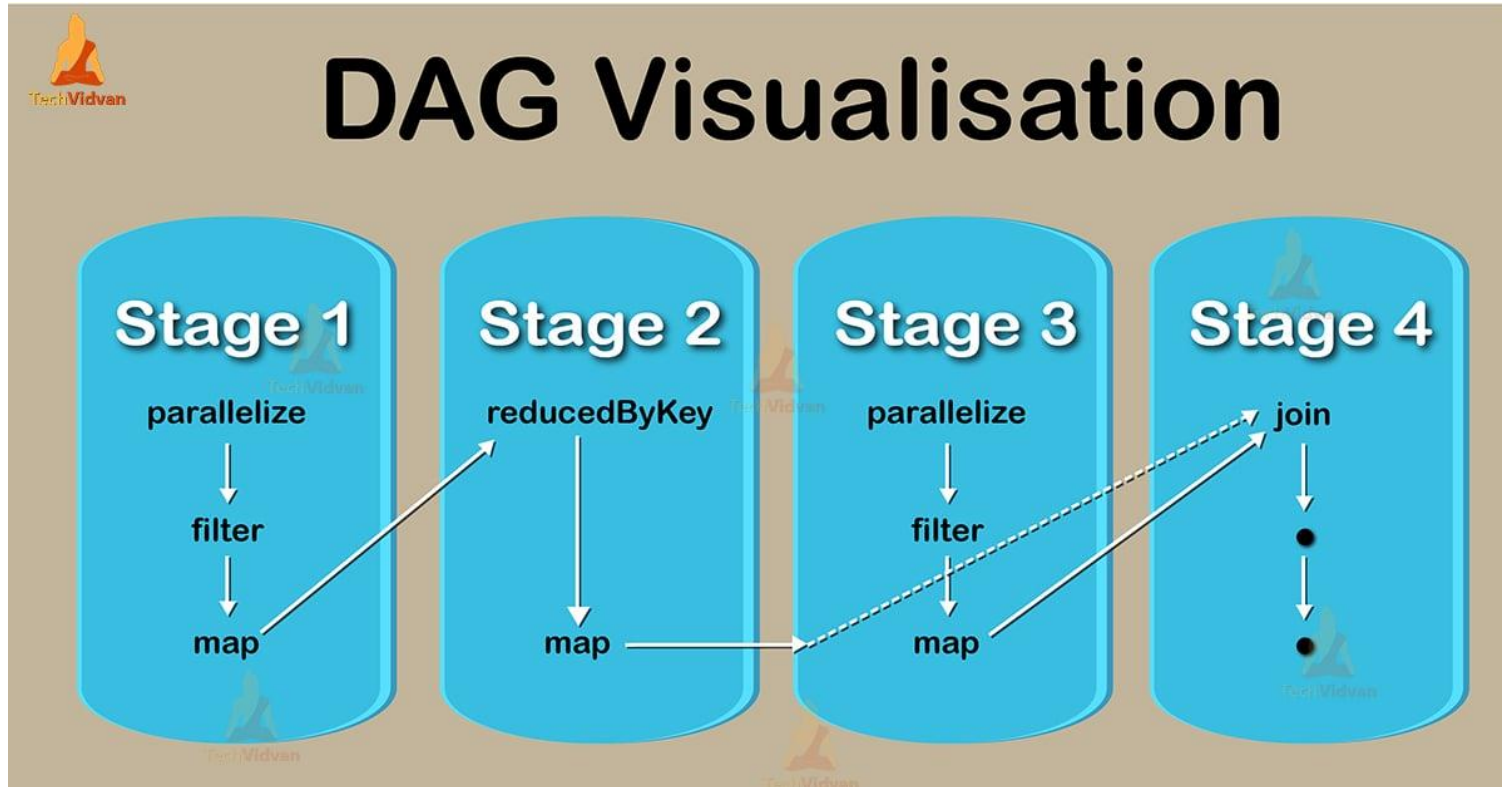
[Source](#)

DAG: Directed Acyclic Graph (Heart of Spark)

- Spark represents each job as a Directed Acyclic Graph (DAG) of operations.
- A DAG describes how data flows through transformations in a Spark application.
- Nodes represent transformations (e.g., filter, map, groupBy).
- Edges represent dependencies between transformations.
- The graph is directed, meaning operations must occur in a specific order.
- The graph is acyclic, meaning it contains no loops.
- Spark builds the DAG before execution instead of running operations immediately.
- Transformations are lazy and only execute when an action is triggered.
- Spark analyzes the DAG to optimize execution, reduce data movement, combine operations into fewer stages, and enable fault recovery by recomputation.
- Unlike Hadoop MapReduce's fixed map → reduce model, Spark's DAG allows flexible, multi-stage execution plans.

The DAG is Spark's execution blueprint.

DAG: Directed Acyclic Graph (Heart of Spark)



<https://techvidvan.com/tutorials/apache-spark-dag-directed-acyclic-graph/>

Resilient Distributed Dataset (RDD)

What Is an RDD in Spark?

- An RDD (Resilient Distributed Dataset) is Spark's original low-level data abstraction for representing a distributed collection of data that can be processed in parallel across a cluster.

Key characteristics

- **Distributed:** Data is split into partitions across nodes
- **Immutable:** Transformations create new RDDs
- **Fault-tolerant:** Lost data can be recomputed using lineage
- **Lazy:** Computation happens only when an action is triggered

How RDDs work

- RDDs track how they were created (their lineage)
- If a partition is lost, Spark recomputes it instead of restoring from disk
- Operations are either:
 - **Transformations** (e.g., map, filter)
 - **Actions** (e.g., count, collect)

Relationship to modern Spark

- RDDs are the foundation of Spark
- Higher-level APIs (DataFrames, Spark SQL) are built on top of RDDs
- Most users interact with DataFrames instead of RDDs today

An RDD is created when Spark defines a distributed dataset during a transformation, and it is used as a node in the DAG that Spark executes when an action triggers computation.

Lazy Evaluation & Optimization

Spark uses lazy evaluation to improve performance.

- Transformations (e.g., select, filter) do not execute immediately
- Spark builds a logical plan instead
- Execution happens only when an action is called

Why this matters

- Enables query optimization
- Avoids unnecessary computation
- Allows Spark to collapse multiple steps into fewer stages

Spark doesn't run your code line-by-line...it builds a plan.

DataFrames define *what* to compute, DAGs define *how* to compute it, and RDDs are the low-level execution units Spark uses under the hood.

Apache Spark Components

<u>Component</u>	<u>What it is</u>	<u>What it does</u>
Spark Core	The foundational execution engine	Handles task scheduling, memory management, fault tolerance, and execution of the DAG across the cluster
Spark SQL	Structured data processing layer	Enables DataFrames and SQL queries; uses the Catalyst optimizer to plan and optimize queries
Structured Streaming	Streaming data engine	Processes continuous data streams using the same APIs as batch jobs, with fault tolerance and exactly-once semantics
MLlib	Distributed machine learning library	Provides scalable implementations of common ML algorithms (classification, regression, clustering, etc.)
GraphX	Graph processing library	Supports graph-based computation such as network analysis using vertices and edges

All of these components share the same execution engine, Spark Core, which is why Spark can handle SQL, ML, and streaming in a unified way.

How Spark Fits into the System

Apache Spark is a compute engine, not a storage system.

- Spark does not permanently store data or manage hardware resources.
- Instead, Spark integrates with other systems to perform distributed computation.

Spark relies on three external layers:

Cluster management: Provides CPU and memory resources;

- Examples: Databricks, YARN, Kubernetes

Storage: Holds durable data outside Spark

- Examples: HDFS, S3, ADLS, GCS

Table & data management: Adds structure and reliability on top of storage

- Examples: Delta Lake, Iceberg, Hudi

Key architectural principle

- Compute and storage are decoupled
- Spark can scale, stop, or restart without data loss

What is PySpark?

- **PySpark is the Python interface for Apache Spark.**
- It allows users to write Spark programs using Python instead of Scala or Java.
- PySpark code defines distributed computations, not local Python operations.
- Python runs on the driver, where it builds the execution plan.
- The actual computation runs in the Spark engine across the cluster.
- Data is processed in parallel on worker nodes, not in the Python interpreter.

Important distinctions

- PySpark is not pandas
- PySpark is not single-machine Python
- PySpark is optimized for large, distributed datasets

PySpark lets Python users control distributed Spark execution.



Hands On.....

PySpark

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

What to work on this week

What To Work On This Week

- Homework #4 (Hands-on Assignment)
- Reading: Chapters 2 & 3 of Data Engineering Fundamentals on O'Reilly
 - Remember to check the syllabus for assigned readings
- Review Back-up slides for further context